

# Assignment 3: Classes & CRT

---

## Test Plan

The primary UART transmitter features verified in this test environment include:

- **Correct Frame Structure:** Verifying that the serialized output encapsulates the payload, beginning with a valid Start bit (0) and terminating with a valid Stop bit (1).
- **Variable Frame Length:** Validating that the transmission frame dynamically adjusts to 10 bits when parity is disabled, and 11 bits when the `parity_en` signal is asserted.
- **LSB-First Transmission:** Confirming that the 8-bit data payload is correctly serialized and shifted out starting with the Least Significant Bit (LSB).
- **Accurate Parity Calculation:** Ensuring the parity bit is mathematically correct based on whether the configuration is set to EVEN or ODD parity.
- **Handshake Signal Integrity:** Guaranteeing that the `tx_busy` signal remains continuously asserted throughout the entire active transmission, as this serves as an important handshaking mechanism when the uart transmitter is integrated with other blocks.

Additionally, robustness and reset assertion testing is performed by asserting an asynchronous reset at random clock cycles during an active transmission. Also, negative testing is performed by injecting random noise into the `data_in`, `tx_start`, and parity configuration signals mid-frame. These stress tests are designed to verify:

- **Asynchronous Reset Assertion & FSM Coverage:** The test environment evaluates the design's ability to immediately terminate an active transmission when `rst_n` is asserted mid-frame. This verifies that the FSM correctly aborts the operation, returns to a stable **IDLE** state, drives the `tx` line high, and is prepared to accept new stimulus. By triggering this reset across various active phases (such as **START**, **DATA**, or **PARITY**), the test systematically exercises all asynchronous state transitions back to **IDLE**, strictly validating the recovery logic and exercising all possible FSM transitions thus drive the FSM code coverage to 100%.
- **FSM Robustness:** The design should properly isolate the active frame from input noise. Variations on the data and start pins must be ignored while the transmitter is busy, ensuring they are only sampled during the **IDLE** state to prevent frame corruption.
- **Configuration Signal Handling:** The test evaluates the FSM's sampling behavior for `parity_en` and `even_parity`. By toggling these inputs mid-transmission, the environment exposes whether these configuration signals are dynamically read at the end of the **DATA** state as expected or they are latched at the beginning of transmission.

To systematically evaluate the DUT, each feature is initially tested in isolation. This targeted approach precisely identifies which specific requirements the design fails to meet before transitioning to integrated, multi-feature testing.

This isolated testing uses constrained randomization to cover a many possible scenarios for a single feature while intentionally masking others. This methodology guarantees that any observed failures are strictly attributed to the targeted requirement, preventing overlapping bugs from hiding the root cause. Additionally, the verification process was coverage driven, meaning the number of test cases was continuously adjusted to ensure 100% functional and code coverage is met.

The specific constraints applied to isolate each feature are:

- **LSB-First Transmission:** A common implementation error is transmitting the payload Most Significant Bit (MSB) first. Because palindromic data reads identically in both directions and will hide this bug, the payload is strictly constrained to be a non-palindrome. To further isolate this test, parity is disabled (`NO_PARITY`) to prevent frame-length or parity-calculation bugs from interfering. (Conversely, when isolating other features, the data is forced to be a palindrome to deliberately bypass the common MSB bug).
- **Variable Frame Length:** To strictly evaluate the FSM's ability to adjust frame sizes, the data payload is constrained as a palindrome. Additionally, the golden model's parity value check is temporarily ignored and not compared to obtained parity bit, ensuring the test only evaluates the structural transition between 10-bit and 11-bit frames.
- **Accurate Parity Calculation:** To investigate the mathematical parity logic, `parity_en` is constrained high and the payload is constrained as a palindrome. Constraints are used to specifically target both EVEN and ODD parity modes to ensure the internal XOR/XNOR logic behaves as specified.
- **Correct Frame Structure:** To verify the fundamental Start and Stop bit framing without interference, the payload is constrained to a palindrome and parity type is constrained to be no parity.

Additionally, corner cases are investigated by constraining the input data to the boundary values, specifically all-ones (`0xFF`) and all-zeros (`0x00`), to stress the FSM's idle-detection logic. Also, the checkerboard patterns like (`0b0101_0101`) and (`0b1010_1010`) were used to ensure maximum toggle activity on the serial output line. This alternating sequence tests the shift register's bit-transition logic and guarantees high toggle coverage across the design's internal data paths.

# Test Environment

## Enum Packages

The enums used in this test environment are collected into a package that is shown below:

```
package enum_pkg;

    typedef enum logic [1:0] {
        ODD,
        EVEN,
        NO_PARITY
    } parity_e;

    typedef enum int {
        NO_NOISE,
        IGNORE_PARITY_BIT,
        DATA_NOISE,
        DATA_NOISE_IGNORE_PARITY,
        TOGGLE_PARITY_EN,
        TOGGLE_PARITY_EN_IGNORE_PARITY,
        TOGGLE_PARITY_TYPE,
        ALL_NOISE,
        TEST_RESET,
        TEST_RESET_IGNORE_PARITY
    } test_type_e;

endpackage
```

Figure 1: Enum package

The `parity_e` configures the parity settings for input stimulus where:

- **ODD**: Sets UART packet object to have an odd parity bit.
- **EVEN**: Sets UART packet object to have an even parity bit.
- **NO\_PARITY**: Sets UART packet object to have no parity bit.

The `test_type_e` configures the type of negative tests performed on the DUT and each type is explained below:

- **NO\_NOISE**: Keeps all inputs of the DUT fixed until the DUT is done with transmission.
- **IGNORE\_PARITY\_BIT**: Same as **NO\_NOISE** test type but doesn't check that the parity bit value is as expected.
- **DATA\_NOISE**: Randomly change `data_in` during transmission to assess if changing input data while DUT is busy will corrupt its output data. Also, to verify FSM robustness, the input signal `tx_start` is randomly toggled mid-transmission to check for unintended frame corruption. This toggling is restricted to the first 9 cycles to ensure it is de-asserted before the **STOP** and **IDLE** states, preventing accidental back-to-back transmissions.
- **DATA\_NOISE\_IGNORE\_PARITY**: Same as **DATA\_NOISE** test type, however, this test type doesn't check that parity bit value is as expected, it is only concerned with frame structure.
- **TOGGLE\_PARITY\_EN**: Toggles `parity_en` once during transmission before entering **PARITY** state to ensure DUT acts according to recent `parity_en` and not according to previous `parity_en` value it latched when it started transmission.
- **TOGGLE\_PARITY\_EN\_IGNORE\_PARITY**: Same as **TOGGLE\_PARITY\_EN** but, this test type doesn't check that the parity bit value is as expected.
- **TOGGLE\_PARITY\_TYPE**: Toggles `even_parity` signal once during transmission before entering **PARITY** state to ensure DUT acts according to recent parity type signal and not according to previous `even_parity` value it latched before starting transmission.
- **ALL\_NOISE**: Randomly varies input data and `tx_start` during transmission and also toggles parity configuration signals `parity_en` and `even_parity`.
- **TEST\_RESET**: Asserts the active-low reset signal (`rst_n`) at a randomized clock cycle during an active transmission. The environment then monitors the output over an 11-cycle observation window after reset is asserted to verify that the DUT immediately aborts the ongoing frame, recovers to the **IDLE** state, maintains a stable high on the `tx` line and a low on the `tx_busy` line.
- **TEST\_RESET\_IGNORE\_PARITY**: Same as **TEST\_RESET** but, this test type doesn't check that the parity bit value is as expected.

## Interface

The interface used for the UART dut and test environment is shown below:

```
interface uart_if#(int PAYLOAD_SIZE)(input logic clk);

    logic                rst_n;
    logic                tx_start;
    logic [PAYLOAD_SIZE-1:0] data_in;
    logic                parity_en;
    logic                even_parity;
    logic                tx;
    logic                tx_busy;

    modport te(input clk, tx, tx_busy,
               output rst_n, tx_start, data_in, parity_en, even_parity);

    modport dut(input clk, rst_n, tx_start, data_in, parity_en, even_parity,
                output tx, tx_busy);

endinterface
```

Figure 2: Interface

## UART Packet Class

### Constraints and Helper Functions

The UART Packet class with its set constraints is shown below in the following snippets:

```
import enum_pkg::*;
class Uart_packet;

    rand logic data_palindrome; //decide if type of data is palindrom or non-palindrome
    rand logic [7:0] data_in;

    rand parity_e parity_type;
    rand logic parity_en;
    rand logic even_parity;

    //----- CONSTRAINTS -----//

    constraint parity_c{
        parity_type dist {
            NO_PARITY := 34,
            EVEN := 33,
            ODD := 33
        };
    }

    constraint palindrome_c {
        data_in[7] == data_in[0];
        data_in[6] == data_in[1];
        data_in[5] == data_in[2];
        data_in[4] == data_in[3];
    }

    constraint non_palindrome_c {
        !((data_in[7] == data_in[0]) &&
          (data_in[6] == data_in[1]) &&
          (data_in[5] == data_in[2]) &&
          (data_in[4] == data_in[3]));
    }

    constraint force_all_ones_c {
        data_in == 8'hFF;
    }

    constraint force_all_zeros_c {
        data_in == 8'h00;
    }

endclass
```

Figure 3: Uart packet class properties declarations and constrains

```

constraint force_checkerboard_c {
    (data_in == 8'hAA) || (data_in == 8'h55);
}

constraint force_no_parity_c {
    parity_type == NO_PARITY;
}
constraint force_parity_c {
    parity_type != NO_PARITY;
}
constraint force_even_parity_c {
    parity_type == EVEN;
}
constraint force_odd_parity_c {
    parity_type == ODD;
}

function void post_randomize();
    if(parity_type == NO_PARITY) begin
        parity_en = 0;
    end
    else if(parity_type == EVEN) begin
        parity_en = 1;
        even_parity = 1;
    end
    else begin
        parity_en = 1;
        even_parity = 0;
    end
endfunction

```

Figure 4: Remaining of Uart packet class constrains and functions

```

//----- FUNCTIONS & TASKS -----//
function new();
    uart_cg = new();
endfunction

function void display();
    $display("Sending >>> data_in = %8b, parity mode = %s",data_in,parity_type.name() );
endfunction

```

Figure 5: Remaining of Uart packet class functions

## Cover Group

The cover-group used in the UART packet class that is used to check coverage of stimulus is shown below:

```
//----- COVER GROUPS -----//
covergroup uart_cg;
  cp_parity: coverpoint parity_type{
    bins no_parity = {NO_PARITY};
    bins even_parity = {EVEN};
    bins odd_parity = {ODD};
  }

  cp_data_auto: coverpoint data_in{
    option.auto_bin_max = 256;
  }

  cp_data: coverpoint data_in{
    bins all_zeros = {'h00};
    bins all_ones = {'hFF};
    bins checkerboard = {'h55, 'hAA};

    bins low_range = {[8'h01:8'h7E]};
    bins high_range = {[8'h7F:8'hFE]};

    bins data_even_parity = {[0:255]} with (^item == 0);
    bins data_odd_parity = {[0:255]} with (^item == 1);
  }

  cross_parity_data: cross cp_parity, cp_data{
    option.cross_auto_bin_max = 0; // disable automatic creation of all other bins to improve performance
    bins even_parity_data_no_parity = binsof(cp_parity.no_parity) && binsof(cp_data.data_even_parity);
    bins even_parity_data_even_parity = binsof(cp_parity.even_parity) && binsof(cp_data.data_even_parity);
    bins even_parity_data_odd_parity = binsof(cp_parity.odd_parity) && binsof(cp_data.data_even_parity);

    bins odd_parity_data_no_parity = binsof(cp_parity.no_parity) && binsof(cp_data.data_odd_parity);
    bins odd_parity_data_even_parity = binsof(cp_parity.even_parity) && binsof(cp_data.data_odd_parity);
    bins odd_parity_data_odd_parity = binsof(cp_parity.odd_parity) && binsof(cp_data.data_odd_parity);
  }
endgroup
```

Figure 6: Uart cover-group

The coverage model is structurally divided to track parity configurations, data variations, and the intersections between them:

- **Parity Configuration (cp\_parity):** Tracks the randomized parity\_type variable to guarantee that the NO\_PARITY, EVEN, and ODD operational modes are all successfully exercised by the environment.
- **Payload Exhaustion & Corner Cases (cp\_data\_auto & cp\_data):** To fully test the shift register, two data cover-points are utilized. An auto-binned cover-point (auto\_bin\_max = 256) is used to prove that every single possible 8-bit payload (0x00 to 0xFF) is generated and transmitted. A secondary cover-point explicitly tracks important corner cases (all-zeros, all-ones, and checkerboard transitions), while also categorizing the payloads by their inherent mathematical parity (having an even or odd number of '1' bits).
- **Logic Cross Coverage (cross\_parity\_data):** The physical parity bit transmitted by the DUT depends on two factors: the configured parity mode and the inherent mathematical parity of the payload. This cross ensures all 6 logical permutations are explicitly verified (e.g., EVEN parity mode computing an even-parity payload vs. an odd-parity payload). This guarantees that the RTL's internal XOR/XNOR calculation trees are stressed under all scenarios.

## Variable Declarations and Instantiation

The variable and object declarations are shown in the snippets below.

```
`timescale 1ns/1ps
`include "Uart_packet.sv"

module uart_tx_te(uart_if.te uart_tx_if);
    //----- PARAMETERS -----//
    int MAX_BUSY_CYCLES = 100;
    int RESET_WINDOW    = 11;
    //----- STIMULUS DECLARATIONS -----//
    Uart_packet uart_packet_i;
    logic [7:0] expected_data_in;
    //----- OUTPUT DATA DECLARATIONS -----//
    typedef struct packed{
        logic tx;
        logic tx_busy;
    } data_out_t;

    data_out_t data_out_queue[$];

    //----- EXPECTED DATA ARRAY DECLARATION -----//
    data_out_t expected_out_queue[$];

    //----- TEST INFO -----//
    int config_no      = 0;
    int tests_passed   = 0;
    int tests_failed   = 0;
    int tests_invalid  = 0;
    int total_tests    = 100;
    int reset_cycle    = 20;

    //----- RUNNING TESTCASES -----//
    string verb; //configure details printed from test bench
    string tb_state;
    int show_per_test;
    bit pkt_state;
    test_type_e test_config;
```

Figure 7: Variable and object declarations



## Random Stimulus Generation

The function generating random stimulus by randomizing the Uart packet object under set constraints is shown below:

```
//----- Generating random stimulus -----//
function automatic bit generate_stimulus(ref Uart_packet uart_packet_i);
    if(uart_packet_i.randomize()) begin
        uart_packet_i.post_randomize();
        if(show_per_test == 0 || show_per_test == 2) uart_packet_i.display();
        return 1;
    end
    else begin
        $display("Randomization failed :(");
        return 0;
    end
endfunction
```

Figure 8: Stimulus generation function

## Golden Model

The golden model function that generates the expected frame based on input data data\_in, parity configurations and test type. It is shown in the snippets below:

```
//----- Golden model -----//
function automatic void golden_model(ref Uart_packet pkt, test_type_e test_type, int reset_cycle);
    data_out_t exp_out;
    bit parity_state;
    bit parity_toggle;
    bit parity_type_toggle;
    bit ignore_parity;
    bit test_reset;
    int cycles_no;

    parity_toggle = ((test_type == TOGGLE_PARITY_EN_IGNORE_PARITY) || (test_type == TOGGLE_PARITY_EN) || (test_type == ALL_NOISE));
    parity_type_toggle = ((test_type == TOGGLE_PARITY_TYPE) || (test_type == ALL_NOISE));
    parity_state = ((parity_toggle)?(pkt.parity_en):(pkt.parity_en));
    ignore_parity = (test_type == TOGGLE_PARITY_EN_IGNORE_PARITY) || (test_type == IGNORE_PARITY_BIT) || (test_type == DATA_NOISE_IGNORE_PARITY);
    ignore_parity = ignore_parity || (test_type == TEST_RESET_IGNORE_PARITY);
    test_reset = (test_type == TEST_RESET) || (test_type == TEST_RESET_IGNORE_PARITY);
    cycles_no = 0;

    if(test_reset && cycles_no == reset_cycle) begin
        exp_out = '{tx:1'b1, tx_busy: 1'b0};
    end
    else begin
        exp_out = '{tx:1'b0, tx_busy: 1'b1};
    end
    cycles_no++;
    expected_out_queue.push_back(exp_out); //start bit

    for(int i = 0; i < uart_tx_if.PAYLOAD_SIZE; i++) begin
        if(test_reset && cycles_no == reset_cycle) begin
            exp_out = '{tx:1'b1, tx_busy: 1'b0};
            expected_out_queue.push_back(exp_out);
            cycles_no++;
            break;
        end
        if(test_reset && cycles_no > reset_cycle) begin
            break;
        end
        exp_out = '{tx:pkt.data_in[i], tx_busy: 1'b1};
        expected_out_queue.push_back(exp_out);
        cycles_no++;
    end
end
```

Figure 9: Golden model function

```

if(test_reset && cycles_no == reset_cycle) begin
    exp_out = '{tx:1'b1,tx_busy: 1'b0};
    expected_out_queue.push_back(exp_out);
    cycles_no++;
end else if (parity_state && (!test_reset || (test_reset && cycles_no < reset_cycle))) begin
    if(ignore_parity) begin
        exp_out = '{tx:1'bx,tx_busy: 1'b1};
        expected_out_queue.push_back(exp_out);
    end
    else if(pkt.even_parity ^ parity_type_toggle) begin
        exp_out = '{tx:~(^pkt.data_in), tx_busy: 1'b1};
        expected_out_queue.push_back(exp_out);
    end
    else begin
        exp_out = '{tx: ^(pkt.data_in),tx_busy: 1'b1};
        expected_out_queue.push_back(exp_out);
    end
    cycles_no++;
end

if(test_reset && cycles_no == reset_cycle) begin
    exp_out = '{tx: 1'b1,tx_busy: 1'b0};
    expected_out_queue.push_back(exp_out); //idle
    cycles_no++;
end else if(!test_reset || cycles_no < reset_cycle) begin
    exp_out = '{tx: 1'b1,tx_busy: 1'b1};
    expected_out_queue.push_back(exp_out); //stop bit
    cycles_no++;
end

if(!test_reset) begin
    exp_out = '{tx:1'b1,tx_busy: 1'b0};
    expected_out_queue.push_back(exp_out); //idle
end

while(test_reset && cycles_no < (reset_cycle + RESET_WINDOW)) begin
    exp_out = '{tx:1'b1,tx_busy: 1'b0};
    expected_out_queue.push_back(exp_out); //idle
    cycles_no ++;
end
endfunction

```

Figure 10: Remaining of the golden model function

The transmitted parity bit depends on the live values of the configuration signals (`parity_en` and `even_parity`). To accurately reflect this behavior during noise-injection tests where parity configuration signals get toggled during transmission, the golden model dynamically updates its expected frame structure and recalculates the parity bit using the newly toggled values of these signals.

As parity input signals are configuration signals the parity bit is calculated based on recent values of `parity_en` and `even_parity`. So the golden model includes parity bit and calculates it based on inverted values of `parity_en` and `even_parity` for test configurations that toggle the parity signals during transmission.

For test configurations that ignore the value of the parity bit it is set to don't care `1'bx` so it matches any value set for the parity bit.

For **TEST\_RESET**, the golden model dynamically constructs the expected output by tracking the number of clock cycle of the transmission. Prior to the targeted reset cycle, it populates the expected output queue with standard UART protocol elements. Once the cycle counter matches the programmed reset cycle, the model immediately aborts normal frame generation and injects the default idle state (`tx = 1`, `tx_busy = 0`), padding the queue with these idle values for the entire duration of the post-reset observation window.

## Driver

The task used to drive DUT is shown below:

```
//----- Driving dut -----//
task automatic drive_stim(ref Uart_packet pkt);
    uart_tx_if.data_in = pkt.data_in;
    uart_tx_if.parity_en = pkt.parity_en;
    uart_tx_if.even_parity = pkt.even_parity;
    uart_tx_if.tx_start = 1'b1;
    @(negedge uart_tx_if.clk);
    uart_tx_if.tx_start = 1'b0;
    pkt.uart_cg.sample();
endtask
```

Figure 11: Driving DUT task

This task drives input data to DUT and sets `tx_start` to HIGH for one cycle to ensure DUT picks up input data and parity configurations and begins transmission. It then de-asserts `tx_start` to LOW to ensure frames aren't transmitted back to back so each frame can be captured separately and get compared to expected.

## Monitor

The monitor task is shown below.

```
//----- Collecting output -----//
task automatic collect_output_data(ref Uart_packet pkt, test_type_e t);
    data_out_t data_out;
    int cycles_no;
    int toggle_parity;
    toggle_parity = $urandom_range(7,1);
    cycles_no = 0;
    while(tx_busy_tb) begin
        if(t == DATA_NOISE || t == ALL_NOISE || (t == DATA_NOISE_IGNORE_PARITY)) begin
            data_in_tb = $urandom;
            if(cycles_no < 9) tx_start_tb = $urandom;
            else tx_start_tb = 0;
        end
        if((t == TOGGLE_PARITY_EN) || (t == TOGGLE_PARITY_EN_IGNORE_PARITY) || (t == ALL_NOISE)) begin
            if(cycles_no == toggle_parity) begin
                parity_en_tb = ~parity_en_tb;
            end
        end
        if(t == TOGGLE_PARITY_TYPE || (t == ALL_NOISE)) begin
            if(cycles_no == toggle_parity) begin
                even_parity_tb = ~even_parity_tb;
            end
        end
        @(negedge clk);
        data_out = '{tx: tx_tb,
                    tx_busy: tx_busy_tb
                    };
        data_out_queue.push_back(data_out);
        cycles_no++;
        if(cycles_no == MAX_BUSY_CYCLES) begin
            $display("Transmitter is busy for too long, ... stopping collecting outputs");
            break;
        end
    end
endtask
```

Figure 12: Collector task

The output collector task continuously samples the serial output as long as the `tx_busy` signal remains HIGH, capturing the transmitted frame. To prevent simulation hangs however, it implements a timer that forcefully breaks the sampling loop if the DUT remains busy for an abnormally large number of cycles. This timeout mechanism effectively detects and reports critical "stuck-at-1" bugs within the FSM's busy logic.

The collector task also handles driving DUT with noise weather in input data, `tx_start` signal or by toggling parity configuration signals during transmission in accordance to test type used. This task ensures noise is injected within an appropriate margin, where `tx_start` can only be asserted within the first 9 cycles to ensure no accidental back to back transmissions occur. Also parity configurations are toggled within first 7 cycles of transmission to ensure toggled value is set before DUT finishes **DATA** state and enters **PARITY** state.

Moreover, the collector acts as a synchronized event driver for reset testing. It maintains an internal cycle counter, which the task uses to track the progression of the frame and dynamically asserts the active-low reset signal (`rst_n = 0`) at the moment the elapsed cycles match the randomly generated `reset_cycle` target.

## Checker

The checker task is shown below. It compares expected transmitted frame stored in queue to obtained frame from DUT.

```
//----- Checking results -----//
task automatic check_results(const ref data_out_t expected_out_queue[$], const ref data_out_t data_out_queue[$]);
    bit queues_match;
    string queue_type;
    queues_match = 1;
    if(expected_out_queue.size() != data_out_queue.size()) begin
        queues_match = 0;
    end else begin
        foreach(expected_out_queue[i]) begin
            if(i >= data_out_queue.size()) begin
                queues_match = 0;
                break;
            end
            if(expected_out_queue[i] != data_out_queue[i])begin
                queues_match = 0;
                break;
            end
        end
    end
    end if(!queues_match) begin
        tests_failed = tests_failed + 1;
        if(show_per_test == 0 || show_per_test == 2) begin
            $error("[FAILED]");
            queue_type = "Expected";
            display_queue(expected_out_queue,queue_type);

            queue_type = "Obtained";
            display_queue(data_out_queue,queue_type);
        end
    end else begin
        tests_passed = tests_passed + 1;
        if(show_per_test == 0) begin
            $display("[PASSED :])");
            queue_type = "Expected";
            display_queue(expected_out_queue,queue_type);

            queue_type = "Obtained";
            display_queue(data_out_queue,queue_type);
        end
    end
endtask
```

Figure 13: Checker

## Additional Helper Functions and Tasks

The tasks used to print summary of tests and reset test configuration is shown below. The reset task turns off all conflicting constraints on the uart packet object before randomization to ensure randomization is done successfully and then desired constraints are selected for each test type.

```
//----- Print Test bench summary -----//
task print_summary(input int config_no);
    $display("----- test bench summary for configuration number: %0d----- ", config_no);
    $display("tests failed: %0d out of %0d ", tests_failed,total_tests);
    $display("invalid tests: %0d out of %0d", tests_invalid,total_tests);
    $display("tests passed: %0d out of %0d", tests_passed,total_tests);
endtask

//----- Reset test values -----//
task automatic reset(ref Uart_packet uart_packet_i);
    tests_failed = 0;
    tests_invalid = 0;
    tests_passed = 0;
    expected_out_queue.delete();
    data_out_queue.delete();

    uart_packet_i.palindrome_c.constraint_mode(0);
    uart_packet_i.non_palindrome_c.constraint_mode(0);
    uart_packet_i.force_no_parity_c.constraint_mode(0);
    uart_packet_i.force_parity_c.constraint_mode(0);
    uart_packet_i.force_even_parity_c.constraint_mode(0);
    uart_packet_i.force_odd_parity_c.constraint_mode(0);
    uart_packet_i.force_all_ones_c.constraint_mode(0);
    uart_packet_i.force_all_zeros_c.constraint_mode(0);
    config_no++;
endtask
```

Figure 14: Print summary and reset tasks

The task used to print frame from expected or output data queue is shown below.

```
//----- Print frame -----//
task automatic display_queue(const ref data_out_t queue[$], const ref string queue_type);
    $display("%s number of bits in frame = %0d",queue_type, queue.size());

    $write ("%s frame data = ",queue_type);
    foreach(queue[i]) begin
        $write("%b",queue[i].tx);
        if(i%4 == 3 && (i < (queue.size() - 1))) $write("_");
    end
    $display();

    $write ("%s frame tx busy = ",queue_type);
    foreach(queue[i]) begin
        $write("%b",queue[i].tx_busy);
        if(i%4 == 3 && (i < (queue.size() - 1)) ) $write("_");
    end
    $display();
endtask
```

Figure 15: Print frame function

The tasks used to run tests is shown in the snippet below.

```
//----- Run tests -----//
task automatic run_tests(ref Uart_packet uart_packet_i, test_type_e test_config);
repeat(total_tests) begin
    pkt_state = generate_stimulus(uart_packet_i);
    if(!pkt_state) begin
        $display("Randomization failed... running tests is terminated");
        break;
    end
    if(test_config == TEST_RESET || test_config == TEST_RESET_IGNORE_PARITY) begin
        reset_cycle = $urandom_range(0,10);
    end else begin
        reset_cycle = 20;
    end
    if(show_per_test == 0 || show_per_test == 2) $display("the cycle at which reset is asserted = %0d",reset_cycle);
    golden_model(uart_packet_i,test_config,reset_cycle);
    drive_stim(uart_packet_i);
    collect_output_data(uart_packet_i,test_config,reset_cycle);
    check_results(expected_out_queue, data_out_queue);
    @(negedge uart_tx_if.clk);
    uart_tx_if.rst_n = 1'b1;
    @(negedge uart_tx_if.clk);
    expected_out_queue.delete();
    data_out_queue.delete();
end
print_summary(config_no);
endtask
```

Figure 16: Run tests task

The function used to determine level of verbosity is shown in the snippet below.

```
//----- Determine Level of detail printed in testbench -----//
function automatic int verb_value(string verb, ref string tb_state);
    int ans;
    if(verb == "summary") begin
        ans = 1;
        tb_state = "SUMMARY";
    end
    else if(verb == "error") begin
        ans = 2;
        tb_state = "DISPLAYING ERRORS ONLY";
    end
    else if(verb == "detailed") begin
        ans = 0;
        tb_state = "DETAILED";
    end
    else begin
        ans = 0;
        tb_state = "DETAILED";
    end
    return ans;
endfunction
```

Figure 17: Determine verbosity function

## The Initial Block

Snippet of the initial block are shown below.

```
initial begin
  if(! $value$plusargs("verbosity=%s", verb)) begin
    verb = "detailed";
  end
  show_per_test = verb_value(verb,tb_state);
  $display("-----");
  $display("----- UART-TX TESTBENCH (%s) -----",tb_state);
  $display("-----");

  //deassert the reset signal
  rst_n = 0;
  @(negedge clk);
  rst_n = 1;
  @(negedge clk);
  uart_packet_i = new();

  // ----- Ensure correct frame structure -----//
  reset(uart_packet_i);
  total_tests = 100;
  $display("----- Running %0d tests to ensure correct frame structure -----",total_tests);
  uart_packet_i.palindrome_c.constraint_mode(1);
  uart_packet_i.force_no_parity_c.constraint_mode(1);

  test_config = NO_NOISE;
  run_tests(uart_packet_i, test_config);

  // ----- Ensure correct frame structure while ignoring parity bit -----//
  reset(uart_packet_i);
  total_tests = 100;
  $display("----- Running %0d tests to ensure correct frame structure while ignoring parity bit -----",total_tests);
  uart_packet_i.palindrome_c.constraint_mode(1);

  test_config = IGNORE_PARITY_BIT;
  run_tests(uart_packet_i, test_config);
```

Figure 18: Snippet of the initial block

```
// ----- Ensure that parity bit is set correctly -----//
reset(uart_packet_i);
total_tests = 100;
$display("----- Running %0d tests to stress test parity bit is set correctly -----",total_tests);
uart_packet_i.palindrome_c.constraint_mode(1);
uart_packet_i.force_parity_c.constraint_mode(1);

test_config = NO_NOISE;
run_tests(uart_packet_i, test_config);

// ----- Ensure that parity bit is set correctly for odd parity -----//
reset(uart_packet_i);
total_tests = 100;
$display("----- Running %0d tests to stress test parity bit is set correctly for odd parity -----",total_tests);
uart_packet_i.palindrome_c.constraint_mode(1);
uart_packet_i.force_odd_parity_c.constraint_mode(1);

test_config = NO_NOISE;
run_tests(uart_packet_i, test_config);

// ----- Ensure that parity bit is set correctly for even parity -----//
reset(uart_packet_i);
total_tests = 100;
$display("----- Running %0d tests to stress test parity bit is set correctly for even parity -----",total_tests);
uart_packet_i.palindrome_c.constraint_mode(1);
uart_packet_i.force_even_parity_c.constraint_mode(1);

test_config = NO_NOISE;
run_tests(uart_packet_i, test_config);

// ----- Mixture of different types of payloads and parity settings -----//
reset(uart_packet_i);
total_tests = 100;
$display("----- Running %0d tests of random payloads and parity settings -----",total_tests);
test_config = NO_NOISE;
run_tests(uart_packet_i, test_config);
```

Figure 19: Another snippet of the initial block

## Dut Wrapper

The wrapper for the DUT is shown below

```
module uart_tx_wrapper(uart_if.dut uart_tx_if);

    uart_tx dut(.clk(uart_tx_if.clk),
                .rst_n(uart_tx_if.rst_n),
                .tx_start(uart_tx_if.tx_start),
                .data_in(uart_tx_if.data_in),
                .parity_en(uart_tx_if.parity_en),
                .even_parity(uart_tx_if.even_parity),
                .tx(uart_tx_if.tx),
                .tx_busy(uart_tx_if.tx_busy)
    );

endmodule
```

Figure 20: Wrapper that takes DUT interface and connect it to the DUT

## Top Module

The top module is shown below:

```
`timescale 1ns/1ps
`define PAYLOAD_SIZE 8

module top();
    //----- PARAMETERS -----//
    int CLOCK_PERIOD = 10;

    //----- CLOCK GENERATION -----//
    logic clk = 0;
    logic rst_n = 0;

    always begin
        #(CLOCK_PERIOD/2) clk = ~clk;
    end

    //----- INTERFACE INSTANTIATION -----//
    uart_if #(PAYLOAD_SIZE(`PAYLOAD_SIZE)) uart_tx_if(clk);

    //----- DUT INSTANTIATION -----//
    uart_tx_wrapper dut(uart_tx_if.dut);

    //----- TESTBENCH INSTANTIATION -----//
    uart_tx_te uart_te(uart_tx_if.te);

endmodule
```

Figure 21: Top Module



# Coverage Results

## Functional Coverage

The functional coverage results for every bins are provided below. As shown in the coverage extraction below, the test-bench successfully achieved 100% functional coverage. All defined bins across the cover-points and crosses were hit, confirming that every intended parity configuration and data scenario was exercised.

| Name                             | Class Type | Coverage | Goal | % of Goal | Status | Included | Merge_instances | Get_inst_coverage | Comment |
|----------------------------------|------------|----------|------|-----------|--------|----------|-----------------|-------------------|---------|
| /uart_tx_te_sv_unit/Uart_packet  |            | 100.00%  |      |           |        |          |                 |                   |         |
| TYPE uart_cg                     |            | 100.00%  | 100  | 100.00... |        | ✓        |                 |                   | auto(1) |
| CVP uart_cg::cp_parity           |            | 100.00%  | 100  | 100.00... |        | ✓        |                 |                   |         |
| bin no_parity                    |            | 880      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin even_parity                  |            | 846      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin odd_parity                   |            | 826      | 1    | 100.00... |        | ✓        |                 |                   |         |
| CVP uart_cg::cp_data_auto        |            | 100.00%  | 100  | 100.00... |        | ✓        |                 |                   |         |
| CVP uart_cg::cp_data             |            | 100.00%  | 100  | 100.00... |        | ✓        |                 |                   |         |
| bin all_zeros                    |            | 92       | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin all_ones                     |            | 77       | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin checkerboard                 |            | 66       | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin low_range                    |            | 1158     | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin high_range                   |            | 1225     | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin data_even_parity             |            | 1829     | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin data_odd_parity              |            | 723      | 1    | 100.00... |        | ✓        |                 |                   |         |
| CROSS uart_cg::cross_parity_data |            | 100.00%  | 100  | 100.00... |        | ✓        |                 |                   |         |
| bin even_parity_data_no_parity   |            | 652      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin even_parity_data_even_parity |            | 600      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin even_parity_data_odd_parity  |            | 577      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin odd_parity_data_no_parity    |            | 228      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin odd_parity_data_even_parity  |            | 246      | 1    | 100.00... |        | ✓        |                 |                   |         |
| bin odd_parity_data_odd_parity   |            | 249      | 1    | 100.00... |        | ✓        |                 |                   |         |

Figure 22: Functional coverage results

## Code Coverage

### Statement Coverage

The statement code coverage result is provided below. As shown in the coverage extraction below, 100% statement coverage is achieved showing that the test-bench exercised every line in the DUT.

| Statement Coverage: |      |      |        |          |
|---------------------|------|------|--------|----------|
| Enabled Coverage    | Bins | Hits | Misses | Coverage |
| -----               | ---- | ---- | -----  | -----    |
| Statements          | 25   | 25   | 0      | 100.00%  |

=====Statement Details=====

Statement Coverage for instance /top/dut/dut --

| Line           | Item | Count | Source |
|----------------|------|-------|--------|
| ----           | ---- | ----- | -----  |
| File uart_tx.v |      |       |        |
| 24             | 1    | 34152 |        |
| 26             | 1    | 301   |        |
| 27             | 1    | 301   |        |
| 28             | 1    | 301   |        |
| 29             | 1    | 301   |        |
| 30             | 1    | 301   |        |
| 31             | 1    | 301   |        |
| 35             | 1    | 7506  |        |
| 36             | 1    | 7506  |        |
| 38             | 1    | 2552  |        |
| 39             | 1    | 2552  |        |
| 40             | 1    | 2552  |        |
| 41             | 1    | 2552  |        |
| 42             | 1    | 2552  |        |
| 47             | 1    | 2542  |        |
| 48             | 1    | 2542  |        |
| 52             | 1    | 19868 |        |
| 53             | 1    | 19868 |        |
| 54             | 1    | 19868 |        |
| 57             | 1    | 1527  |        |
| 59             | 1    | 900   |        |
| 64             | 1    | 1524  |        |
| 65             | 1    | 1524  |        |
| 69             | 1    | 2411  |        |
| 70             | 1    | 2411  |        |

Figure 23: Statement coverage results

## Branch Coverage

The branch code coverage result is provided below. As shown in the coverage report snippet below, the DUT achieved 100% branch coverage showing that the test-bench exercised every branch in the DUT.

```
Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses  Coverage
  -----
  Branches              15        15         0    100.00%

=====Branch Details=====

Branch Coverage for instance /top/dut/dut

  Line      Item      Count      Source
  ----      -
  File uart_tx.v
  -----IF Branch-----
    25              34152      Count coming in to IF
    25          1          301
    32          1        33851
Branch totals: 2 hits of 2 branches = 100.00%

  -----CASE Branch-----
    33              33851      Count coming in to CASE
    34          1          7506
    46          1          2542
    51          1        19868
    63          1          1524
    68          1          2411
Branch totals: 5 hits of 5 branches = 100.00%

  -----IF Branch-----
    37              7506      Count coming in to IF
    37          1          2552
    37          1          4954      All False Count
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
    40              2552      Count coming in to IF
    40          1          1253
    40          2          1299
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
    55              19868      Count coming in to IF
    55          1          2427
    55          1          17441      All False Count
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
    56              2427      Count coming in to IF
    56          1          1527
    58          1          900
Branch totals: 2 hits of 2 branches = 100.00%
```

Figure 24: Branch coverage results

## Condition Coverage

The condition code coverage result is provided below. As shown in the coverage report snippet below, the DUT achieved 100% condition coverage showing that the test-bench has set every sub boolean statement to true or false in conditional statements of the DUT.

```
Condition Coverage:
  Enabled Coverage      Bins   Covered   Misses   Coverage
  -----
  Conditions            1      1         0   100.00%

=====Condition Details=====

Condition Coverage for instance /top/dut/dut --

  File uart_tx.v
  -----Focused Condition View-----
Line      55 Item    1 (bit_cnt == 7)
Condition totals: 1 of 1 input term covered = 100.00%

Expression Coverage:
  Enabled Coverage      Bins   Covered   Misses   Coverage
  -----
  Expressions           3      3         0   100.00%
```

Figure 25: Condition coverage results

## Toggle Coverage

The toggle code coverage result is provided below. As shown in the coverage report snippet below, the DUT achieved 100% toggle coverage showing that the test-bench toggled every bit in the DUTs' signals both from 0 to 1 and from 1 to 0.

```
Toggle Coverage:
  Enabled Coverage      Bins   Hits     Misses   Coverage
  -----
  Toggles              62     62         0   100.00%

=====Toggle Details=====

Toggle Coverage for instance /top/dut/dut --

Node      1H->0L      0L->1H  "Coverage"
-----
Total Node Count    =      31
Toggled Node Count  =      31
Untoggled Node Count =       0

Toggle Coverage     =   100.00% (62 of 62 bins)
```

Figure 26: Toggle coverage results

## Expression Coverage

The expression code coverage result is provided below. As shown in the coverage report snippet below, the DUT achieved 100% expression coverage showing that the test-bench exercised all possible combinations of inputs in DUT expressions.

```
Expression Coverage:
  Enabled Coverage
  -----
  Expressions      Bins   Covered   Misses   Coverage
                    ---
                    3      3         0    100.00%

=====Expression Details=====

Expression Coverage for instance /top/dut/dut --

  File uart_tx.v
  -----Focused Expression View (Bimodal)-----
Line      40 Item    1 (even_parity? ^data_in: ~^data_in)
Expression totals: 3 of 3 input terms covered = 100.00%
```

Figure 27: Expression coverage results

## FSM Coverage

The FSM coverage result is provided below. As shown in the coverage report snippet below, the DUT achieved 100% FSM coverage showing that the test-bench exercised all the possible FSM states and state transitions within the DUT.

```
FSM Coverage:
  Enabled Coverage      Bins      Hits      Misses  Coverage
  -----
  FSM States            5        5         0    100.00%
  FSM Transitions       9        9         0    100.00%

=====FSM Details=====

FSM Coverage for instance /top/dut/dut --

FSM_ID: state
  Current State Object : state
  -----
  State Value MapInfo :
  -----
Line      State Name      Value
-----
  34      IDLE             0
  46      START            1
  51      DATA            2
  68      STOP             4
  63      PARITY           3
  Covered States :
  -----
      State      Hit_count
      -----
      IDLE       2712
      START      2552
      DATA      2542
      STOP       2424
      PARITY     1527
  Covered Transitions :
  -----
Line      Trans_ID      Hit_count      Transition
-----
  42        0          2552      IDLE -> START
  48        1          2542      START -> DATA
  26        2           10      START -> IDLE
  59        3          900      DATA -> STOP
  57        4          1527      DATA -> PARITY
  26        5           115      DATA -> IDLE
  70        6          2424      STOP -> IDLE
  65        7          1524      PARITY -> STOP
  26        8           3       PARITY -> IDLE

Summary
-----
  FSM States            5        5         0    100.00%
  FSM Transitions       9        9         0    100.00%
```

Figure 28: FSM coverage results

# Output Waveforms and Printed Values

The summary of each test configuration is shown in the snippets below.

```
# -----
# ----- UART-TX TESTBENCH (SUMMARY) -----
# -----
# ----- Running 100 tests to ensure correct frame structure -----
# ----- test bench summary for configuration number: 1-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
# ----- Running 100 tests to ensure correct frame structure while ignoring parity bit -----
# ----- test bench summary for configuration number: 2-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
# ----- Running 350 tests to stress test that the payload is sent in correct bit order (LSB first) -----
# ----- test bench summary for configuration number: 3-----
# tests failed: 350 out of 350
# invalid tests: 0 out of 350
# tests passed: 0 out of 350
# ----- Running 100 tests to stress test parity bit is set correctly -----
# ----- test bench summary for configuration number: 4-----
# tests failed: 100 out of 100
# invalid tests: 0 out of 100
# tests passed: 0 out of 100
# ----- Running 100 tests to stress test parity bit is set correctly for odd parity -----
# ----- test bench summary for configuration number: 5-----
# tests failed: 100 out of 100
# invalid tests: 0 out of 100
# tests passed: 0 out of 100
# ----- Running 100 tests to stress test parity bit is set correctly for even parity -----
# ----- test bench summary for configuration number: 6-----
# tests failed: 100 out of 100
# invalid tests: 0 out of 100
# tests passed: 0 out of 100
```

Figure 29: Summary for test configurations 1-6

```
# ----- Running 500 tests of random payloads and parity settings -----
# ----- test bench summary for configuration number: 7-----
# tests failed: 491 out of 500
# invalid tests: 0 out of 500
# tests passed: 9 out of 500
# ----- Running 1 tests of no parity with payload = 0xFF corner case -----
# ----- test bench summary for configuration number: 8-----
# tests failed: 0 out of 1
# invalid tests: 0 out of 1
# tests passed: 1 out of 1
# ----- Running 10 tests of parity type with payload = 0xFF corner case -----
# ----- test bench summary for configuration number: 9-----
# tests failed: 0 out of 10
# invalid tests: 0 out of 10
# tests passed: 10 out of 10
# ----- Running 10 tests of random parity settings with payload = 0xFF corner case -----
# ----- test bench summary for configuration number: 10-----
# tests failed: 4 out of 10
# invalid tests: 0 out of 10
# tests passed: 6 out of 10
# ----- Running 1 tests of no parity with payload = 0x00 corner case -----
# ----- test bench summary for configuration number: 11-----
# tests failed: 0 out of 1
# invalid tests: 0 out of 1
# tests passed: 1 out of 1
# ----- Running 10 tests of parity type with payload = 0x00 corner case -----
# ----- test bench summary for configuration number: 12-----
# tests failed: 0 out of 10
# invalid tests: 0 out of 10
# tests passed: 10 out of 10
# ----- Running 10 tests of random parity settings with payload = 0x00 corner case -----
# ----- test bench summary for configuration number: 13-----
# tests failed: 8 out of 10
# invalid tests: 0 out of 10
# tests passed: 2 out of 10
# ----- Running 100 tests to ensure data and tx_start are latched correctly -----
# ----- test bench summary for configuration number: 14-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
```

Figure 30: Summary for test configurations 7-14

```

# ----- Running 100 tests to ensure data and tx_start are latched correctly for palindromes while ignoring parity -----
# ----- test bench summary for configuration number: 15-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
# ----- Running 100 tests to ensure data and tx_start are latched correctly while ignoring parity -----
# ----- test bench summary for configuration number: 16-----
# tests failed: 92 out of 100
# invalid tests: 0 out of 100
# tests passed: 8 out of 100
# ----- Running 100 tests to ensure data and tx_start are latched correctly under different parity and payload settings -----
# ----- test bench summary for configuration number: 17-----
# tests failed: 98 out of 100
# invalid tests: 0 out of 100
# tests passed: 2 out of 100
# ----- Running 100 tests to assess impact of toggling parity enable mid transmission while ignoring parity bit value -----
# ----- test bench summary for configuration number: 18-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
# ----- Running 100 tests to assess impact of toggling parity enable mid transmission -----
# ----- test bench summary for configuration number: 19-----
# tests failed: 96 out of 100
# invalid tests: 0 out of 100
# tests passed: 4 out of 100
# ----- Running 100 tests to assess impact of toggling parity type mid transmission -----
# ----- test bench summary for configuration number: 20-----
# tests failed: 0 out of 100
# invalid tests: 0 out of 100
# tests passed: 100 out of 100
# ----- Running 100 tests to assess impact of toggling parity type mid transmission for different payload types -----
# ----- test bench summary for configuration number: 21-----
# tests failed: 97 out of 100
# invalid tests: 0 out of 100
# tests passed: 3 out of 100
# ----- Running 80 tests for different types of noise sent -----
# ----- test bench summary for configuration number: 22-----
# tests failed: 0 out of 80
# invalid tests: 0 out of 80
# tests passed: 80 out of 80

```

Figure 31: Summary for test configurations 15-22

```

# ----- Running 80 tests for different types of noise sent for different payload types -----
# ----- test bench summary for configuration number: 23-----
# tests failed: 75 out of 80
# invalid tests: 0 out of 80
# tests passed: 5 out of 80
# ----- Running 50 tests of random parity settings with checkerboard corner case -----
# ----- test bench summary for configuration number: 24-----
# tests failed: 50 out of 50
# invalid tests: 0 out of 50
# tests passed: 0 out of 50
# ----- Running 50 tests of setting rst_n to low mid transmission with no parity -----
# ----- test bench summary for configuration number: 25-----
# tests failed: 0 out of 50
# invalid tests: 0 out of 50
# tests passed: 50 out of 50
# ----- Running 50 tests of setting rst_n to low mid transmission while ignoring parity bit -----
# ----- test bench summary for configuration number: 26-----
# tests failed: 0 out of 50
# invalid tests: 0 out of 50
# tests passed: 50 out of 50
# ----- Running 50 tests of setting rst_n to low mid transmission while ignoring parity bit -----
# ----- test bench summary for configuration number: 27-----
# tests failed: 40 out of 50
# invalid tests: 0 out of 50
# tests passed: 10 out of 50

```

Figure 32: Summary for test configurations 23-27

According to the summary, tests of correct frame structure, immunity to data noise and parity toggling and reset assertion mid transmission passed while tests checking correct transmission order and parity bit value failed. So we will investigate the output and waveforms of each test configuration separately to locate the design bugs.



The printed outputs of the first configuration with palindrome data and no parity is shown below:

```
# Sending >>> data_in = 01011010, parity mode = NO_PARITY
# ** Error: [PASSED :)]
#   Time: 12880 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 424
# Expected number of bits in frame = 11
# Expected frame data = 0010_1101_011
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0010_1101_011
# Obtained frame tx busy = 1111_1111_110
```

Figure 33: Printed values of test with palindrome data and no parity

The waveform of a test case with palindrome data and no parity is shown below:

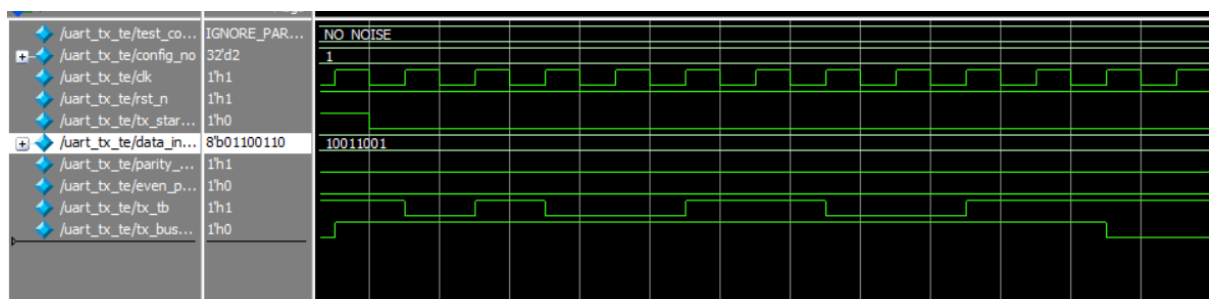


Figure 34: Waveform of test-case with palindrome data and no parity

As shown in above waveform and printed output the design does sent frame in correct structure. The printed outputs of the configuration with palindrome data and parity is shown below:

```
# Sending >>> data_in = 11111111, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 13430 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 424
# Expected number of bits in frame = 12
# Expected frame data = 0111_1111_1x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0111_1111_1111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 11011011, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 13570 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 424
# Expected number of bits in frame = 12
# Expected frame data = 0110_1101_1x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0110_1101_1011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 00011000, parity mode = EVEN
# ** Error: [PASSED :)]
```

Figure 35: Printed values of test with palindrome data and parity

The waveform of a test case with palindrome data and odd parity is shown below:

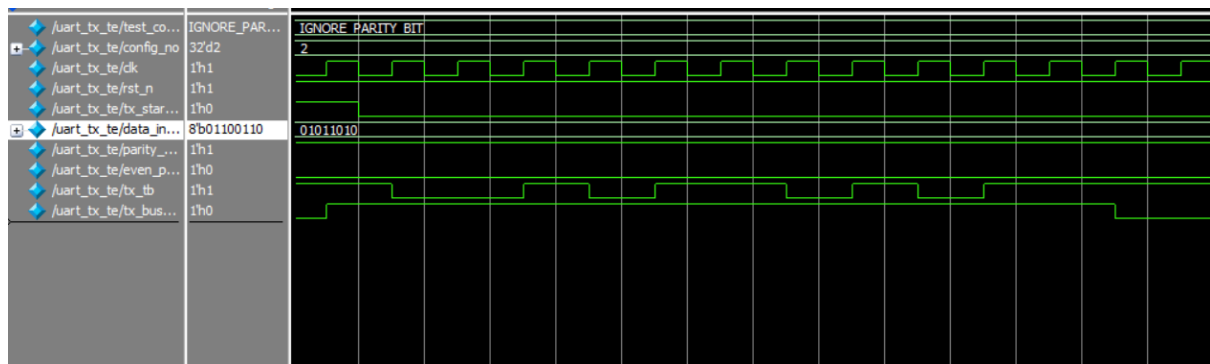


Figure 36: Waveform of test-case with palindrome data and odd parity

The waveform of a test case with palindrome data and even parity is shown below:

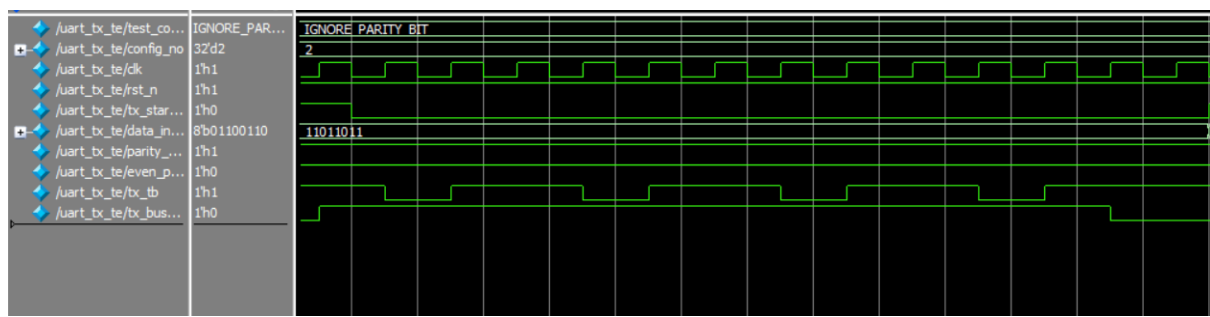


Figure 37: Waveform of test-case with palindrome data and even parity

As shown in above waveform and printed output the design can correctly send frames of variable length according to parity enable.

The printed outputs of the configuration with non-palindrome data and no parity is shown below:

```
# Sending >>> data_in = 00111000, parity mode = ODD
# ** Error: [FAILED]
#   Time: 13260 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0000_1110_0x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0001_1100_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 11101100, parity mode = ODD
# ** Error: [FAILED]
#   Time: 13400 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0001_1011_1x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0111_0110_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 10101110, parity mode = NO_PARITY
# ** Error: [FAILED]
#   Time: 13530 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 11
# Expected frame data = 0011_1010_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0101_0111_011
# Obtained frame tx busy = 1111_1111_110
# Sending >>> data_in = 11011010, parity mode = NO_PARITY
# ** Error: [FAILED]
#   Time: 13660 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 11
# Expected frame data = 0010_1101_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0110_1101_011
# Obtained frame tx busy = 1111_1111_110
```

Figure 38: Printed values of test with non-palindrome data and no parity

The waveform of a test case with non-palindrome and odd parity is shown below:

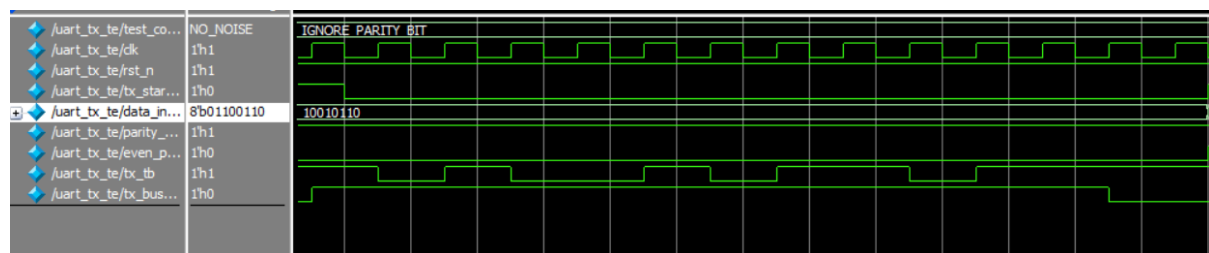


Figure 39: Waveform of test-case with non-palindrome data and no parity

By analyzing the mismatch between expected and obtained frames it is found that data is sent serially in MSB first order rather than LSB first order causing tests with data constrained to non-palindromes to fail.

The printed outputs of the configuration with non-palindrome data and parity and is set to check parity bit value is shown below:

```
# Sending >>> data_in = 11100111, parity mode = ODD
# ** Error: [FAILED]
#   Time: 27240 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0111_0011_1011
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0111_0011_1111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 01000010, parity mode = EVEN
# ** Error: [FAILED]
#   Time: 27380 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0010_0001_0111
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0010_0001_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 01100110, parity mode = EVEN
# ** Error: [FAILED]
#   Time: 27520 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0011_0011_0111
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0011_0011_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 01100110, parity mode = ODD
# ** Error: [FAILED]
#   Time: 27660 ns  Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 416
# Expected number of bits in frame = 12
# Expected frame data = 0011_0011_0011
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0011_0011_0111
# Obtained frame tx busy = 1111_1111_1110
```

Figure 40: Printed values of tests with palindrome data and parity that assess the calculated parity bit

The waveform of a test case with palindrome and odd parity is shown below:

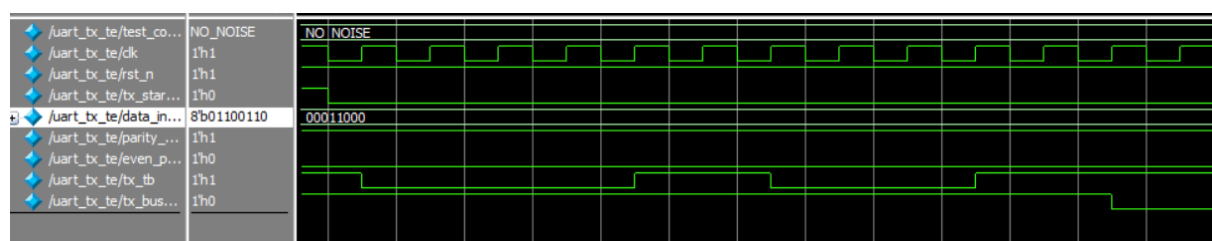


Figure 41: Waveform of test-case with palindrome data and parity

By inspecting the difference between expected and obtained frames it is found that parity bit is toggled where for even parity the design produces parity bit that was supposed to be set for odd parity setting and vice versa.

The printed outputs of the configuration with palindrome data and includes addition of noise in input data and tx\_start during transmission is shown below:

```
# Sending >>> data_in = 01111110, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 12580 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0011_1111_0x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0011_1111_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 10000001, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 12720 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0100_0000_1x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0100_0000_1011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 10000001, parity mode = NO_PARITY
# ** Error: [PASSED :)]
#   Time: 12850 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 11
# Expected frame data = 0100_0000_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0100_0000_111
# Obtained frame tx busy = 1111_1111_110
# Sending >>> data_in = 01100110, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 12990 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0011_0011_0x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0011_0011_0111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 11111111, parity mode = NO_PARITY
# ** Error: [PASSED :)]
#   Time: 13120 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 11
# Expected frame data = 0111_1111_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0111_1111_111
# Obtained frame tx busy = 1111_1111_110
```

Figure 42: Printed values of tests with palindrome data that asses impact of data noise on output frame

The waveform of a test case with palindrome and data noise is shown below:

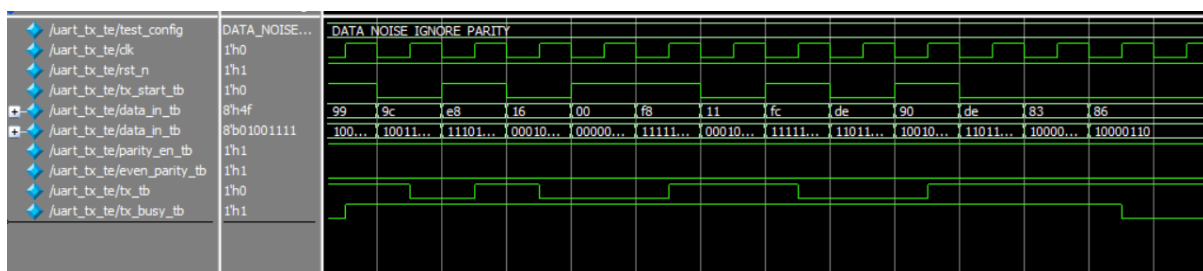


Figure 43: Waveform of test-case with palindrome data and data noise

The above waveform and printed output show design is immune to data noise.

The printed outputs of the configuration with palindrome data and toggles `parity_en` during transmission is shown below:

```
# Sending >>> data_in = 10011001, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 15090 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 11
# Expected frame data = 0100_1100_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0100_1100_111
# Obtained frame tx busy = 1111_1111_110
# Sending >>> data_in = 11000011, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 15220 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 11
# Expected frame data = 0110_0001_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0110_0001_111
# Obtained frame tx busy = 1111_1111_110
# Sending >>> data_in = 00011000, parity mode = NO_PARITY
# ** Error: [PASSED :)]
#   Time: 15360 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0000_1100_0x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0000_1100_0111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 00000000, parity mode = NO_PARITY
# ** Error: [PASSED :)]
#   Time: 15500 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0000_0000_0x11
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0000_0000_0111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 10111101, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 15630 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 11
# Expected frame data = 0101_1110_111
# Expected frame tx busy = 1111_1111_110
# Obtained number of bits in frame = 11
# Obtained frame data = 0101_1110_111
# Obtained frame tx busy = 1111_1111_110
```

Figure 44: Printed values of tests with palindrome data that asses impact of toggling parity enable during transmission

The waveform of a test case with palindrome and parity enable toggling from 1 to 0 is shown below:

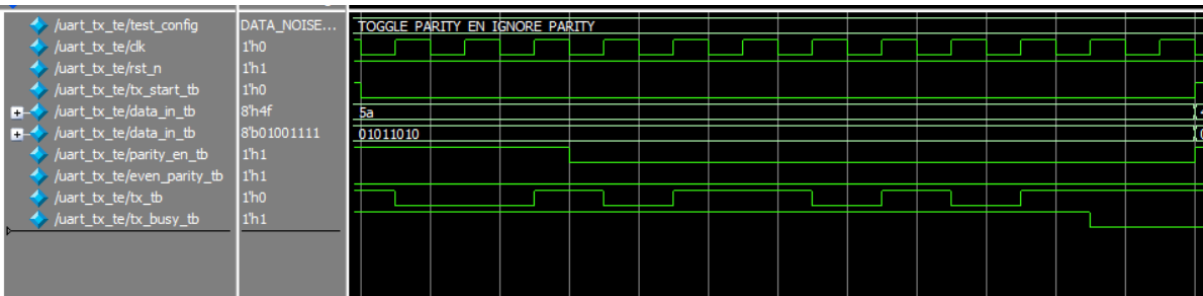


Figure 45: Waveform of test-case with palindrome data and toggled parity enable from high to low

The waveform of a test case with palindrome and parity enable toggling from 0 to 1 is shown below:

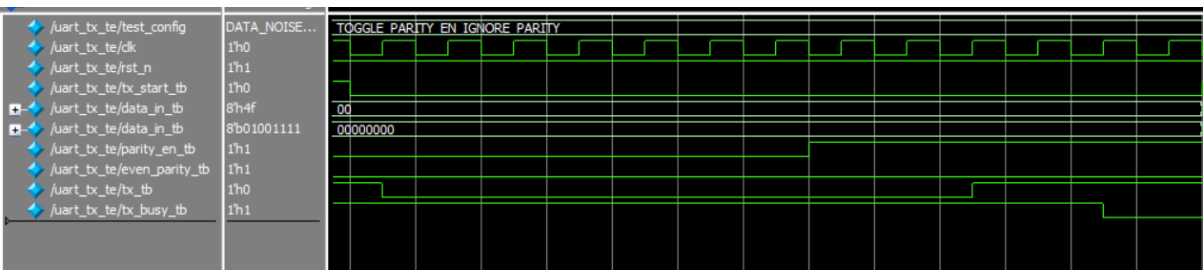


Figure 46: Waveform of test-case with palindrome data and toggled parity enable from high to low

The above waveform and printed output show design is does not latch on parity enable and correctly picks correct parity enable signal.



The printed outputs of the configuration with palindrome data and toggles even\_parity during transmission is shown below:

```
# Sending >>> data_in = 01011010, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 40200 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0010_1101_0111
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0010_1101_0111
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 10100101, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 40340 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0101_0010_1011
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0101_0010_1011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 01011010, parity mode = EVEN
# ** Error: [PASSED :)]
#   Time: 40480 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0010_1101_0011
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0010_1101_0011
# Obtained frame tx busy = 1111_1111_1110
# Sending >>> data_in = 11000011, parity mode = ODD
# ** Error: [PASSED :)]
#   Time: 40620 ns   Scope: uart_tx_te.check_results File: uart_tx_te.sv Line: 435
# Expected number of bits in frame = 12
# Expected frame data = 0110_0001_1111
# Expected frame tx busy = 1111_1111_1110
# Obtained number of bits in frame = 12
# Obtained frame data = 0110_0001_1111
# Obtained frame tx busy = 1111_1111_1110
```

Figure 47: Printed values of tests with palindrome data that asses impact of toggling parity type during transmission

The waveform of a test case with palindrome and parity type toggling from odd to even is shown below:

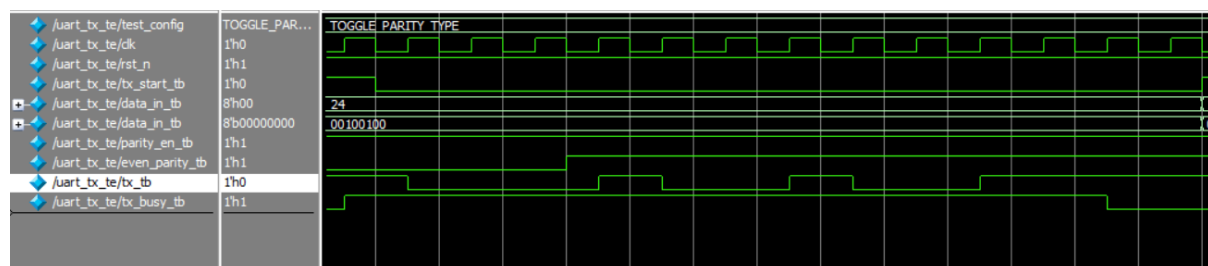


Figure 48: Waveform of test-case with palindrome data and toggled parity type from odd to even



The waveform of a test case with palindrome and parity type toggling from even to odd is shown below:

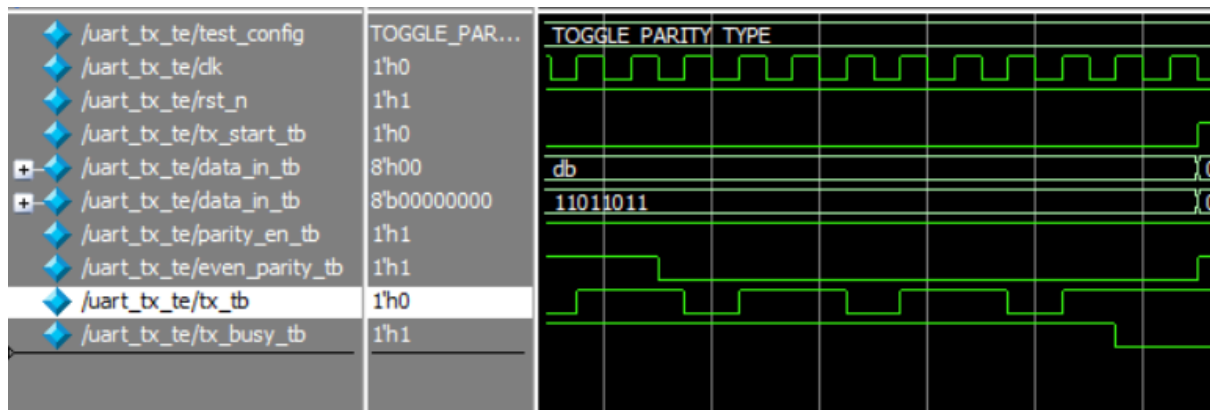


Figure 49: Waveform of test-case with palindrome data and toggled parity type from even to odd

While log shows that tests pass, as the design inverts the parity bit then the parity bit it calculates initially aligns with expected parity bit after the parity type is toggling, which suggests design doesn't check parity type at the end of **DATA** state and instead latches its value.

The printed outputs of the configuration with palindrome data and asserts `rst_n` during transmission is shown below:

```
# Sending >>> data_in = 00111100, parity mode = NO_PARITY
# the cycle at which reset is asserted = 2
# [PASSED :)]
# Expected number of bits in frame = 13
# Expected frame data = 0011_1111_1111_1
# Expected frame tx busy = 1100_0000_0000_0
# Obtained number of bits in frame = 13
# Obtained frame data = 0011_1111_1111_1
# Obtained frame tx busy = 1100_0000_0000_0
# Sending >>> data_in = 10111101, parity mode = ODD
# the cycle at which reset is asserted = 7
# [PASSED :)]
# Expected number of bits in frame = 18
# Expected frame data = 0101_1111_1111_1111_11
# Expected frame tx busy = 1111_1110_0000_0000_00
# Obtained number of bits in frame = 18
# Obtained frame data = 0101_1111_1111_1111_11
# Obtained frame tx busy = 1111_1110_0000_0000_00
# Sending >>> data_in = 00111100, parity mode = NO_PARITY
# the cycle at which reset is asserted = 0
# [PASSED :)]
# Expected number of bits in frame = 11
# Expected frame data = 1111_1111_111
# Expected frame tx busy = 0000_0000_000
# Obtained number of bits in frame = 11
# Obtained frame data = 1111_1111_111
# Obtained frame tx busy = 0000_0000_000
# Sending >>> data_in = 11011011, parity mode = EVEN
# the cycle at which reset is asserted = 2
# [PASSED :)]
# Expected number of bits in frame = 13
# Expected frame data = 0111_1111_1111_1
# Expected frame tx busy = 1100_0000_0000_0
# Obtained number of bits in frame = 13
# Obtained frame data = 0111_1111_1111_1
# Obtained frame tx busy = 1100_0000_0000_0
# Sending >>> data_in = 01011010, parity mode = NO_PARITY
# the cycle at which reset is asserted = 7
# [PASSED :)]
# Expected number of bits in frame = 18
# Expected frame data = 0010_1101_1111_1111_11
# Expected frame tx busy = 1111_1110_0000_0000_00
# Obtained number of bits in frame = 18
# Obtained frame data = 0010_1101_1111_1111_11
# Obtained frame tx busy = 1111_1110_0000_0000_00
```

Figure 50: Printed values of tests with palindrome data that asserts reset mid transmission to ensure DUT returns immediately to **IDLE** state

The waveform of a test case where reset is asserted while DUT is in the **START** state is shown below:

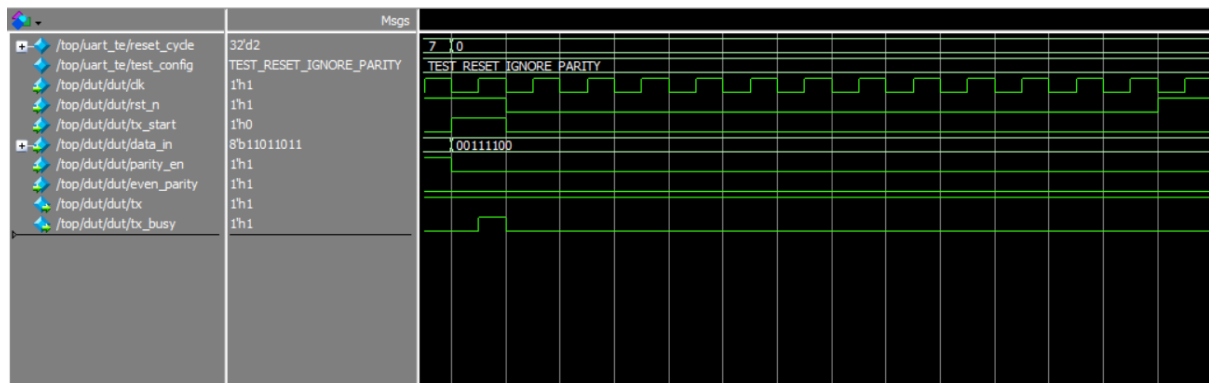


Figure 51: Waveform of test-case where reset is asserted while DUT is in **START** state

The waveform of a test case where reset is asserted while DUT is in the **DATA** state is shown below:

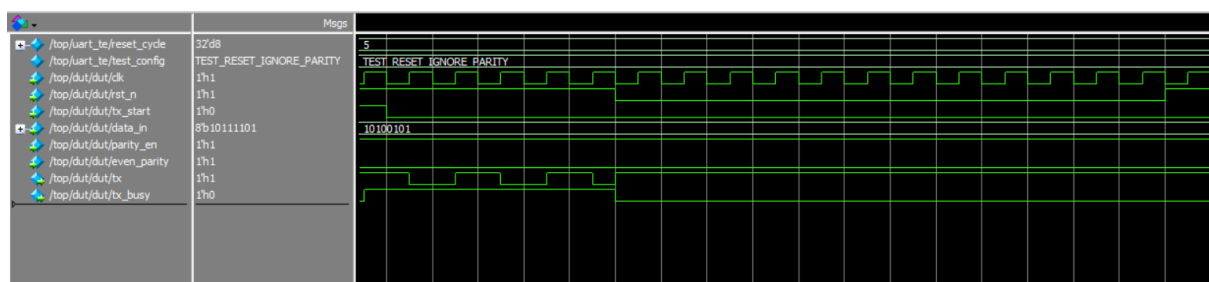


Figure 52: Waveform of test-case where reset is asserted while DUT is in **DATA** state

The waveform of a test case where reset is asserted while DUT is in the **PARITY** state is shown below:

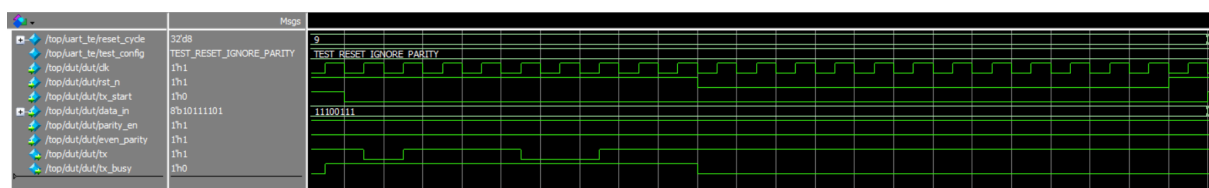


Figure 53: Waveform of test-case where reset is asserted while DUT is in **PARITY** state

The analysis of the simulation logs and the accompanying waveforms confirms that the design correctly responds to the assertion of the `rst_n` signal. When the active-low reset (`rst_n`) is asserted during the **START**, **DATA**, or **PARITY** states, the DUT immediately and asynchronously terminates the transmission, correctly defaulting the **IDLE** state outputs to `tx = 1` and `tx_busy = 0`.

## Reported Bugs

By analyzing the test cases and waveforms the observed bugs are:

- **Incorrect Data Transmission Order:** Palindrome data passes tests while non palindrome data fails and inspecting outputs confirm that design transmits payload in MSB first order rather than LSB.
- **Incorrect Parity Bit value:** The parity bit in the design is swapped between odd and even parity.
- **Latching of Parity Type Signal:** While the tests for toggling parity type midst transmission passed, the failure of tests checking for correct parity bit value suggest that design mixed up parity bit calculation for odd and even parity so when the parity bit was toggled, the design didn't see the new parity type signal but instead its old incorrect parity bit calculation aligned with the old parity type signal suggesting that the design latched the parity type signal at the start of transmission instead of checking its value at the end of the **DATA** state.

## Do File

The do file used is shown below.

```
if { [info exists 1] } {  
    set v_arg $1  
} else {  
    set v_arg "detailed"  
}  
  
vlib work  
vlog enum_pkg.sv Uart_packet.sv uart_if.sv uart_tx.v uart_tx_te.sv top.sv  
vlog -cover bcefst +covercells uart_tx.v  
vsim -voptargs=+acc work.top +verbosity=$v_arg -cover  
coverage save -onexit cov.ucdb  
  
add wave -r *  
run -all  
coverage exclude -code b -du uart_tx -line 33 -allfalse  
coverage report -details -output cov_report.txt
```

Figure 54: Do file